team18@klh:~$ ./monitor --status

Linux System Information & Resource Monitoring Tool

A multithreaded C utility that reads live CPU, memory, and process data directly from the Linux /proc filesystem

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING • 25CS2104E

Team 18 | Section 6

Thyarla Abhinav (2520030075) • J. Advaith (2520030524) • Papaisetty Shashe Preetham (2520030534)

The Problem & Our Objectives

Tools like top/htop hide how system stats are actually gathered from the kernel

PROBLEM

Standard monitoring utilities abstract away their own internals. Users see CPU% and memory usage, but never how the kernel exposes that state — leaving a gap between using a tool and understanding the OS mechanisms beneath it.

OUR GOAL

Build a transparent, from-scratch monitoring tool that reads /proc directly via raw system calls — making every OS concept behind the numbers fully visible and modifiable.

01

Real-time stats

Retrieve live CPU, memory & process data via low-level file I/O on /proc

02

Concurrent polling

Multithreaded POSIX architecture with mutex-synchronized shared data

03

Clean shutdown

Signal handling (SIGINT/SIGTERM) for safe termination & cleanup

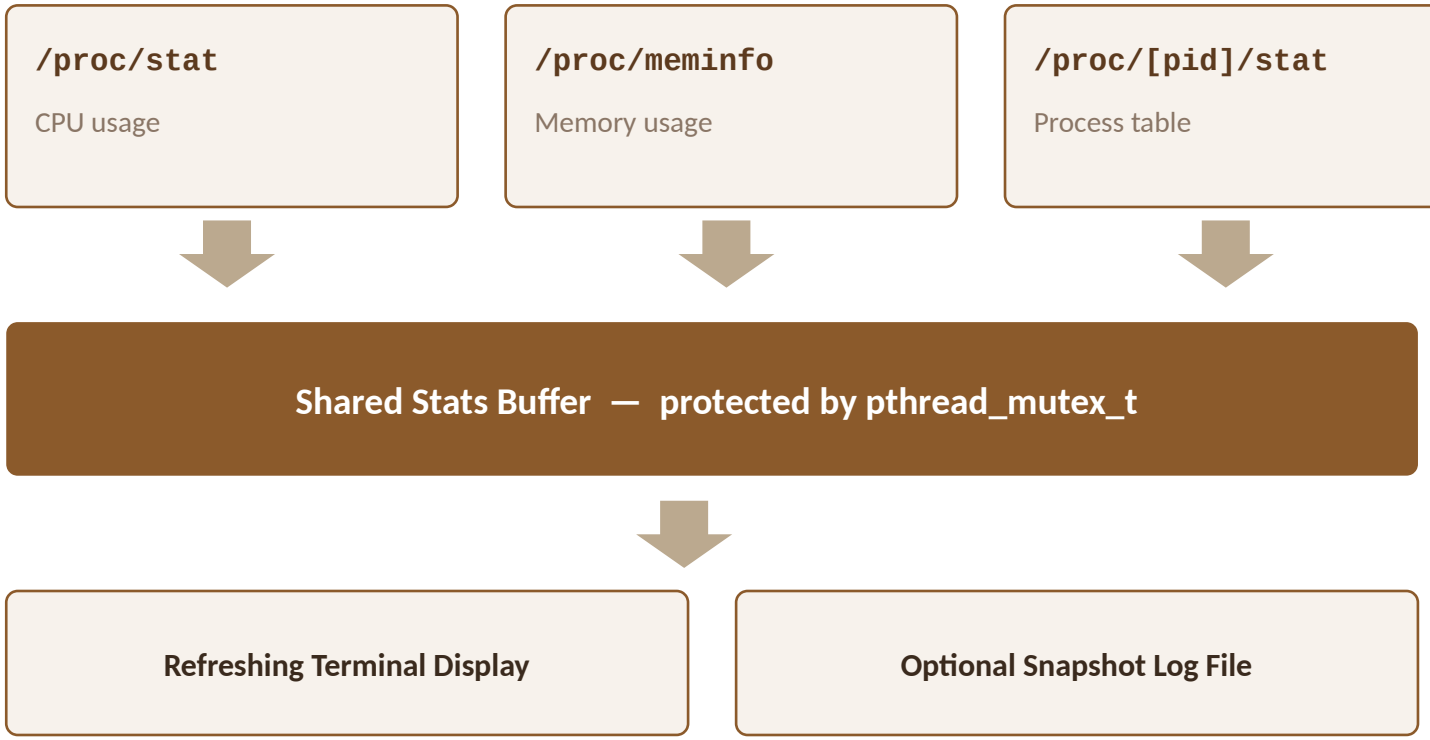
04

Snapshot logging

Optional mode to periodically log system state to disk

System Architecture & Technologies

Worker threads poll /proc concurrently → shared, mutex-protected buffer → display + logging



`SIGINT / SIGTERM → sigaction() handler stops threads, closes file descriptors, exits cleanly`

TECH STACK

Language

`C`

Platform

`Linux / Ubuntu`

Concurrency

`POSIX Threads + Mutex`

Signals

`sigaction(), SIGINT/SIGTERM`

I/O

`open() / read() / close()`

Build

`GCC + Make`

Debug/Analysis

`GDB, strace, Valgrind`

Versioning

`Git & GitHub`

Expected Outcome & CO Mapping

A working command-line resource monitor — comparable to top, built entirely from first principles — that displays live CPU, memory, and a sorted process list, handles concurrent polling safely, shuts down cleanly, and optionally logs snapshots to disk.

COURSE OUTCOME (CO) MAPPING

CO1

Service Layer & Syscalls

Traces the syscall interface: kernel vs user space, /proc as the OS-service abstraction

CO2

Process Control

Extracts PID, PPID & process state via process-control primitives on /proc/[pid]

CO3

Signals

SIGINT/SIGTERM handled with sigaction() for safe, asynchronous shutdown

CO4

Virtual Memory

Reports memory utilization & layout by parsing /proc/meminfo and process memory stats

CO5

File I/O & File Systems

Unbuffered open()/read()/close() calls used to read /proc entries and write logs

CO6

Concurrency

POSIX threads poll resources in parallel, synchronized via pthread_mutex_t